

조인 튜닝1

Simulation Test

모든 문제의 저작권은 아이티파이(ITPE)에 있습니다.

문제의 복제 및 제 3자 공유를 절대 금합니다.



1. Oracle에서 TAB1,TAB2 순으로 NL 조인하도록 유도하고 싶다. 다음 중 (가)안에 넣을 바른 힌트 사용법을 2개 고르시오.

```
SELECT /*+(          가          ) */ .....  
FROM TAB1 A, TAB2 B  
WHERE A, KEY = B, KEY
```

- ① ORDERED USE_NL(B)
- ② ORDERED USE_NL(TAB2)
- ③ LEADING(A) USE_NL(B)
- ④ DRIVING_SITE(A) USE_NL(B)

2. 아래 SQL을 힌트에 지시한 대로 수행할 때, 조건절 비교 순서를 올바르게 나열한 것을 고르시오.

[인덱스 구성]

사원_PK : 사원번호

사원_X1 : 입사일자

고객_PK : 고객번호

고객_X1 : 관리사원번호

```
select /*+ ordered use_n1(c) index(e) index(c) */
```

e. 사원번호, e. 사원명, e. 입사일자, c.고객번호, c.고객명, c.전화번호, c.최종주문금액

from 사원 e, 고객 c

where c. 관리사원번호 = e. 사원번호-----가

and e. 입사일자 >= '20200101' -----나

and e.부서코드 = '10010' -----다

and c. 최종주문금액 >= 20000 -----라

- ① 가 → 나 → 다 → 라
- ② 나 → 다 → 가 → 라
- ③ 나 → 다 → 라 → 가
- ④ 나 → 가 → 다 → 라

3. 다음 중 아래와 같은 SQL에서 가능한 조인 방법을 모두 나열한 것은?

```
SELECT A.ENAME, B.GRADE  
FROM EMP A, SALGRADE B  
WHERE A.SAL BETWEEN B.LOWER_SAL AND B.HIGH_SAL
```

- ① Nested Loop Join
- ② Nested Loop Join, Sort Merge Join
- ③ Sort Merge Join, Hash Join
- ④ Nested Loop Join, Sort Merge Join, Hash Join

4. 아래 SQL은 index 힌트에 인덱스명이나 컬럼명을 지정하지 않았으므로 여러 가지 액세스 경로가 가능하다. 보이는 각 액세스 경로에서 사용하게 될 인덱스를 나열한 것인데, 가장 가능성이 작은 것은?

[인덱스 구성]

사원_PK : 사원번호

사원_X1 : 입사일자

사원_X2 : 부서코드

고객_PK : 고객번호

고객_X1 : 관리사원번호

고객_X2 : 최종주문금액

```
select /*+ ordered use_nl(c) index(e) index(c) */
```

e. 사원번호, e. 사원명, e. 입사일자, c.고객번호, c.고객명, c.전화번호, c.최종주문금액

From 사원 e, 고객 c

where c. 관리사원번호 = e. 사원번호

and e. 입사일자 >= '20200101'

and e.부서코드= '10010'

and c.최종주문금액 >= 20000

- ① 사원_PK, 고객_X1
- ② 사원_X1, 고객_X1
- ③ 사원_X1, 고객_X2
- ④ 사원_X2, 고객_X1

5. 아래 SQL에 대한 튜닝 방안으로 가장 적절한 것을 고르시오.

[인덱스 구성]

상품_PK : 상품코드

상품_X01 : 상품분류코드 + 상품가격 + 공급업체코드

거래_PK : 거래번호

거래_X01 : 거래일자 + 상품코드

거래_X02 : 상품코드 + 거래구분코드 + 거래일자

```
select p. 상품코드, p. 상품가격, t. 거래일자, t. 거래수량, t. 거래금액
from 상품 p, 거래 t
where p. 상품분류코드 = 'KTG'
and p. 공급업체코드 = 'SP83732'
and p. 상품가격 between 10000 and 100000
and t. 상품코드 = p. 상품코드
and t. 거래일자 between ' 20210101' and ' 20210131'
```

Rows	Row Source Operation
368	NESTED LOOPS (cr=1311 pr=124 pw=0 time=1622 us)
69	TABLE ACCESS BY INDEX ROWID 상품 (cr=986 pr=35 pw=0 time=659 us)
69	INDEX RANGE SCAN 상품_X01 (cr=922 pr=31 pw=0 time=645)
368	TABLE ACCESS BY INDEX ROWID 거래 (cr=325 pr=89 pw=0 time=963 us)
385	INDEX RANGE SCAN 거래_X02 (cr=140 pr=21 pw=0 time=326 us)

- ① 상품_X01 인덱스의 컬럼 순서를 조정한다.
- ② 상품_X01 인덱스에 컬럼을 추가한다.
- ③ 거래_X02 인덱스에 컬럼을 추가한다.
- ④ 거래_X02 대신 거래_X01 인덱스를 사용한다.

6. 다음 중 NL 조인의 특징으로 가장 부적절한 것은?

- ① 랜덤 액세스 위주의 조인 방식이다.
- ② 조인할 대상 레코드가 아무리 많아도 ArraySize에 해당하는 최초 N건을 빠르게 출력할 수 있다.
- ③ 인덱스 유무, 인덱스 구성에 의해 성능이 크게 달라진다.
- ④ 부분범위 처리가 가능한 상황에서만 효과적인 조인 방식이다.

7. 다음 두 SQL을 위한 PRA_HST_STC 테이블의 최적 인덱스 구성안을 고르시오.

[SQL 1]

```
SELECT /*+ ORDERED USE_NL(B) */ *  
FROM PRA_HST_STC A, ODM_TRMS B  
WHERE A. SALE_ORG_ID = :saleOrgId  
AND   A.STRD_GRP_ID = B.STRD_GRP_ID  
AND   A.STRD_ID= B.STRD_ID  
ORDER BY A.STC_DT
```

[SQL 2]

```
SELECT /*+ ORDERED USE_NL(B) */ *  
FROM PRA_HST_STC A, ODM_TRMS B  
WHERE A.USER_ID = :userId  
AND   A.SALE_ORG_ID = :saleOrgId  
AND   A.STRD_GRP_ID = B.STRD_GRP_ID  
AND   A.STRD_ID= B.STRD_ID  
ORDER BY A. STC_DT
```

- ① PRA_HST_STC_X1 : SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT
PRA_HST_STC_X2 : USER_ID + SALE_ORG_ID + STRD_GRP_ID + STRD_ID + STC_DT
- ② PRA_HST_STC.X1 : SALE_ORG_ID + USER_ID + STRD_GRP_ID + STRD_ID + STC_DT
- ③ PRA_HST_STC.X1 : SALE_ORG_ID + USER_ID + STC_DT + STRD_GRP_ID + STRD_ID
- ④ PRA_HST_STCX1: SALE_ORG_ID + USER_ID

8. 주문 테이블을 기준으로 고객 테이블과 소트 머지 조인하도록 유도하고자 할 때 빈칸 (가)에 들어갈 오라클 힌트는?

```
select /*+      (가)      */  
      o. 주문번호, o. 고객번호, c. 고객명, c. 전화번호, o. 주문금액  
From 주문 o, 고객 C  
where o.주문일자 >= trunc(sysdate)  
and   c.고객번호 = o.고객번호
```

- ① use_merge (o c)
- ② ordered merge (c)
- ③ ordered use_merge (c)
- ④ leading(o) sort_merge (c)

9. 아래 실행계획이 나타나도록 옵티마이저 힌트 중 가장 적절한 것은?

```
select d. deptno, d.dname, e.empno, e.ename
from emp e, dept d
where d. deptno = e. deptno
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		14	336	8 (25)	00:00:01
1	MERGE JOIN		14	336	8 (25)	00:00:01
2	SORT JOIN		4	44	4 (25)	00:00:01
3	TABLE ACCESS FULL	DEPT	4	44	3 (0)	00:00:01
4	SORT JOIN		14	182	4 (25)	00:00:01
5	TABLE ACCESS FULL	EMP	14	182	3(0)	00:00:01

- ① leading(d) use_merge(e)
- ② ordered sort_merge(e)
- ③ leading(e) use_merge(d)
- ④ ordered sort_merge(d)

10. 다음 중 소트 머지 조인의 특징으로 가장 부적절한 것은?

- ① 랜덤 액세스가 아닌 스캔 위주의 조인 방식이다.
- ② 양쪽 집합을 개별적으로 읽어서 정렬하므로 조인 컬럼에 인덱스가 없어도 상관없다.
- ③ 양쪽 테이블 모두 조인 컬럼에 인덱스가 있으면 따로 정렬을 수행하지 않아도 되므로 매우 유리하다.
- ④ 두 테이블을 각각 읽어 조인 대상 집합을 줄일 수 있을 때 매우 효과적이다.

11. 해시 조인과 비교할 때 소트 머지 조인의 장점을 고르시오.

- ① 메모리를 적게 사용한다.
- ② Temp 테이블스페이스를 적게 사용한다.
- ③ 조인 조건이 '='이 아닐 때도 사용할 수 있다.
- ④ 양쪽 집합이 모두 대용량일 때 해시 조인보다 빠르다.

12. 다음 중 Hash Join에 대한 설명으로 가장 부적절한 것을 2개 고르시오.

- ① 조인 연결고리에 equi-join 조건이 하나라도 있어야 한다.
- ② 일반적으로 큰 집합으로 해시 테이블을 생성하고서 작은 집합을 읽으면서 이 해시 테이블을 탐색하는게 유리하다.
- ③ OLTP 환경에서 수행빈도가 아주 높은 쿼리라도 부담 없이 사용할 수 있다.
- ④ Build Input으로 선택된 집합의 조인 칼럼에는 중복 값이 거의 없어야 효과적이다.

13. 다음의 실행계획이, 나타나도록 유도하고자 할 때, 빈칸 (가)에 들어한 오라클 힌트로 가장 부적절한 것은?

[실행계획]

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		14	364	7 (15)	00:00:01
1	HASH JOIN		14	364	7 (15)	00:00:01
2	TABLE ACCESS FULL	DEPT	4	52	3 (0)	00:00:01
3	TABLE ACCESS FULL	EMP	14	182	7 (15)	00:00:01

[SQL]

```
select /*+      (가)      */ d.deptno, d.dname, e.empno, e.ename
from emp e, dept d
where d.deptno = e. deptno
```

- ① leading(d) use_hash(e)
- ② ordered use_hash(e)
- ③ ordered use_hash(d) swap_join_inputs (d)
- ④ use_hash(e d) swap_join_inputs(d)

14. 다음 중 조인 순서에 가장 영향을 적게 받는 조인 메소드를 고르시오.

- ① NL 조인
- ② 소트 머지 조인
- ③ 해시 조인
- ④ 해시 아우터 조인

15. 아래 SQL에 대한 튜닝 방안으로 가장 부적절한 것은?

[데이터]

대리점 : 1,000개

상품판매실적 : 월평균 100만 건

[인덱스 구성]

대리점_PK : 대리점코드

상품판매실적_PX : 대리점코드 + 상품코드 + 판매일자

상품판매실적_X1 : 판매일자 + 상품코드

```
SELECT A. 대리점명, SUM(B. 매출금액) 매출금액
FROM 대리점 A, 상품판매실적 B
WHERE A. 대리점코드 = B. 대리점코드
AND B. 상품코드 IN ( 'A1847', 'Z04131' )
AND B. 판매일자 BETWEEN ' 20210101' AND '20210331'
GROUP BY B. 대리점코드, A. 대리점명
ORDER BY 1, 2
```

Rows	Row Source Operation
-----	-----
30	SORT GROUP BY
90000	NESTED LOOPS
90000	TABLE ACCESS BY INDEX ROWID 상품판매실적
90000	INDEX RANGE SCAN 상품판매실적_X1 (NONUNIQUE)
90000	TABLE ACCESS BY INDEX ROWID 대리점
90000	INDEX UNIQUE SCAN 대리점_PK (UNIQUE)

- ① 상품판매실적_X1 인덱스를 「 상품코드 + 판매일자 」 순으로 변경한다.
- ② 조인 순서를 변경한다.
- ③ 판매 일자 조건을 만족하는 상품판매실적을 대리점코드로 GROUP BY 한 후 대리점 테이블과 조인하도록 SQL을 변환한다.
- ④ 해시 조인으로 유도한다.

16. 아래 SQL에 대한 튜닝 방안으로 가장 적절한 것을 고르시오

[데이터]

상품 : 50만 rows

상품상세 : 500만 rows

상품변경이력 : 1,000만 rows

[인덱스 구성]

상품_PK : 상품코드

상품상세_PK : 상품코드 + 상품상세코드

상품상세_X1 : 등록일시 + 상품코드

상품변경이력_PK : 상품코드 + 변경일시

```
create table 상품상세_temp
nologging
as
select /*+ ordered use_hash(b) use_hash(c) swap_join_inputs(c) */ *
from 상품상세 a, 상품변경이력 b, 상품 C
where b. 상품코드 = a. 상품코드
and b. 변경일시 like substr(a.등록일시, 1, 8) '%'
and c. 상품코드 = a. 상품코드
```

- ① use_hash(b) 힌트를 use_nl(b)로 수정한다.
- ② ordered use_hash(b) 힌트를 leading(b) use_hash(a)로 수정한다.
- ③ 힌트 전체를 leading(c a b) use_hash(a) use_hash(b) swap_join_inputs(a)로 수정한다.
- ④ 두 번째 조건절을 아래와 같이 수정한다.
and substr(b.변경일시, 1, 8) = substr(a.등록일시, 1, 8)

17. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and    c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0          SELECT STATEMENT Optimizer=ALL_ROWS
1    0      HASH JOIN
2      1    TABLE ACCESS (FULL) OF ' 제품' (TABLE)
3      1    HASH JOIN
4        3  TABLE ACCESS (FULL) OF '구매' (TABLE)
5        3  TABLE ACCESS (FULL) OF '상품' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

18. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and    c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0          SELECT STATEMENT Optimizer=ALL_ROWS
1    0      HASH JOIN
2      1    HASH JOIN
3        2  TABLE ACCESS (FULL) OF '구매' (TABLE)
4        2  TABLE ACCESS (FULL) OF '상품' (TABLE)
5        2  TABLE ACCESS (FULL) OF '제품' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

19. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and    c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0          SELECT STATEMENT Optimizer=ALL_ROWS
1    0      HASH JOIN
2      1    TABLE ACCESS (FULL) OF ' 구매' (TABLE)
3      1    HASH JOIN
4        3  TABLE ACCESS (FULL) OF '제품' (TABLE)
5        3  TABLE ACCESS (FULL) OF '상품' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

20. 아래와 같은 실행계획이 생성되는 힌트 중 가장 적절한 것은?

```
select a.구매번호, a.구매금액, b.제품코드, c.제품명, b.제품수량
from 구매 a, 상품 b, 제품 c
where b. 구매번호 = a. 구매번호
and    c. 제품코드 = b. 제품코드
```

Execution Plan

```
-----
0          SELECT STATEMENT Optimizer=ALL_ROWS
1    0      HASH JOIN
2      1    HASH JOIN
3        2  TABLE ACCESS (FULL) OF '제품' (TABLE)
4        2  TABLE ACCESS (FULL) OF '상품' (TABLE)
5        2  TABLE ACCESS (FULL) OF '구매' (TABLE)
```

- ① /*+ leading(b c a) use_hash(c) use_hash(a) swap_join_inputs(c) */
- ② /*+ leading(c b a) use_hash(b) use_hash(a) swap_join_inputs(a) */
- ③ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(a) */
- ④ /*+ leading(a b c) use_hash(b) use_hash(c) swap_join_inputs(c) */

21. 오라클 11g 이상 버전에서 아래 SQL에 대해 나타날 수 있는 실행계획으로 가장 부적절한 것은?

```
select /*+ ordered use_nl(s) index(p) index(s) */ *  
from 제품 p, 판매 s  
where p.공급코드 = 'P20211212'  
and s.제품코드 = p. 제품코드  
and s.판매일자 between ' 20210101' and '20210131'
```

① Execution Plan

```
-----  
0      SELECT STATEMENT  
1 0      NESTED LOOPS  
2 1        TABLE ACCESS BY INDEX ROWID OF 제품  
3 2          INDEX RANGE SCAN OF 제품_X1  
4 1        TABLE ACCESS BY INDEX ROWID OF 판매  
5 4          INDEX RANGE SCAN OF 판매_X1
```

② Execution Plan

```
-----  
0      SELECT STATEMENT  
1 0      TABLE ACCESS BY INDEX ROWID OF 판매  
2 1        NESTED LOOPS  
3 2          TABLE ACCESS BY INDEX ROWID OF 제품  
4 3            INDEX RANGE SCAN OF 제품_X1  
5 2            INDEX RANGE SCAN OF 판매_X1
```

③ Execution Plan

```
-----  
0      SELECT STATEMENT  
1 0      TABLE ACCESS BY INDEX ROWID OF 제품  
2 1        NESTED LOOPS  
3 2          INDEX RANGE SCAN OF 제품_X1  
4 2          TABLE ACCESS BY INDEX ROWID OF 판매  
5 4            INDEX RANGE SCAN OF 판매_X1
```

④ Execution Plan

```
-----  
0      SELECT STATEMENT  
1 0      NESTED LOOPS  
2 1        NESTED LOOPS  
3 2          TABLE ACCESS BY INDEX ROWID OF 제품  
4 3            INDEX RANGE SCAN OF 제품_X1  
5 2            INDEX RANGE SCAN OF 판매_X1  
6 1          TABLE ACCESS BY INDEX ROWID OF 판매
```

22. 아래 SQL을 위한 ODM_TRMS 테이블의 인덱스 구성에 대한 설명으로 올바른 것은?

[SQL]

```
SELECT /*+ ORDERED USE_L(B) */ *  
  FROM TBL_RPA_HST A, TBL_RPA_INFO B  
 WHERE A.PRA_HIST_ID = :praHistId  
       AND B.RPA_ID = A.RPA_ID  
       AND B.RPA_GRP_ID = :rpaGrpId  
       AND B.PROC_DT BETWEEN :procDt1 AND :procDt2  
 ORDER BY A.RPA_HIST_DT
```

[인덱스 구성]

```
RPA_INFO_IDX_01 : RPA_GRP_ID + PROC_DT + RPA_ID  
RPA_INFO_IDX_02 : RPA_ID + RPA_GRP_ID + PROC_DT  
RPA_INFO_IDX_03 : RPA_GRP_ID + RPA_ID + PROC_DT
```

- ① 상수 또는 변수 조건을 앞에 두는 것이 유리하므로 RPA_INFO_IDX_01 인덱스가 가장 효과적이다.
- ② 조인 조건을 앞에 두는 것이 유리하므로 다른 인덱스보다 RPA_INFO_IDX_02가 효과적이다.
- ③ RPA_INFO_IDX_02, RPA_INFO_IDX_03 인덱스가 최적이며, 둘 간에 성능 차이는 없다.
- ④ RPA_INFO_IDX_01, RPA_INFO_IDX_02, RPA_INFO_IDX_03 인덱스 모두 최적이다.

23. FROM 절에 나열한 주문, 고객, 결제방식 순으로 조인하되, 주문과 고객 테이블은 NL 조인, 결제방식과는 해시 조인하도록 적절한 힌트는?

```
select *  
from 주문 o, 고객 c, 결제방식 t  
where o.주문일자 >= trunc(sysdate)  
and c.고객번호 = 0.고객번호  
and t.결제방식코드 = 0.결제방식코드
```

- ① /*+ ordered use_nl(c) use_hash(t) */
- ② /*+ ordered use_nl(t) use_hash(c) */
- ③ /*+ leading(o c t) use_nl(t) use_hash(c) */
- ④ /*+ leading(c t o) use_nl(c) use_hash(t) */

24. 아래 SQL을 다음과 같은 실행계획이 생성되도록 옵티마이저 힌트를 기술하시오.

[인덱스 구성]

계약_PK : 계약번호

계약_X1 : 계약일자

가입상품_PK : 계약번호 + 상품번호

가입상품_X1 : 가입일자

가입상품정보_PK : 계약번호 + 상품코드 + 상품정보코드

상품_PK : 상품코드

```
select a.계약번호, a.계약명, b.상품코드, b.가입일자, b.할인률, c.상품정보코드, d.상품명
  from 계약 a, 가입상품 b, 가입상품정보 c, 상품 d
 where a.계약일자 = :cntDt
       and b.계약번호 = a.계약번호
       and b.가입일자 = :joinDt
       and c.계약번호 = b.계약번호
       and c.상품코드 = b.상품코드
       and c.상품정보코드 like 'A%'
       and d.상품코드 = c.상품정보코드
```

Execution Plan

```
-----
0          SELECT STATEMENT Optimizer=ALL_ROWS
1      0      HASH JOIN
2      1          TABLE ACCESS (BY INDEX ROWID) OF '상품 ' (TABLE)
3      2          INDEX (RANGE SCAN) OF '상품_PK' (INDEX (UNIQUE))
4      1      NESTED LOOPS
5      4          HASH JOIN
6      5              TABLE ACCESS (BY INDEX ROWID) OF '가입상품' (TABLE)
7      6              INDEX (RANGE SCAN) OF '가입상품_X1' (INDEX)
8      5              TABLE ACCESS (BY INDEX ROWID) OF '계약' (TABLE)
9      8              INDEX (RANGE SCAN) OF '계약_X1' (INDEX)
10     4          INDEX (RANGE SCAN) OF '가입부가상품_PK' (INDEX (UNIQUE))
```

25. 아래는 튜닝이 필요한 SQL에서 스칼라 서브쿼리 부분만 발췌한 것이다. 스칼라 서브쿼리를 수행하는 부분에서 현재의 문제점을 기술하고, 튜닝 방안을 제시하시오

```
SELECT ...
    , (SELECT CASE WHEN A. 일할계산여부 = 'Y'
        THEN NVL(A. 총청구건수, 0) - NVL(A. 청구횟수, 0)
        ELSE B. 할부개월수 - NVL(A. 청구횟수, 0)
        END
    FROM 서비스별할부 A, 할부계획 B
    WHERE A. 서비스계약번호 = AA.서비스계약번호
        AND A. 할부상태코드 = '01'
        AND B. 할부계획ID(+) = A.할부계획ID
        AND ROWNUM <= 1
    ) AS 청구횟수, ...
FROM ( ... ) AA
WHERE ...
```

Id	Operation	Name	Starts	A-Rows	A-Time	Buffers
...	...	...	...	...	...	...
26	COUNT STOPKEY		37243	28953	00:00:34.56	293K
27	NESTED LOOP OUTER		37243	28953	00:00:34.56	293K
28	TABLE ACCESS BY INDEX ROWID	서비스별할부	37243	28953	00:00:17.26	177K
29	INDEX RANGE SCAN	서비스별할부_X1	37243	28953	00:00:08.05	149K
30	TABLE ACCESS BY INDEX ROWID	할부계획	28953	28953	00:00:17.26	115K
31	INDEX UNIQUE SCAN	할부계획_PK	28953	28953	00:00:08.15	86860
...	...	...	...	...	...	...